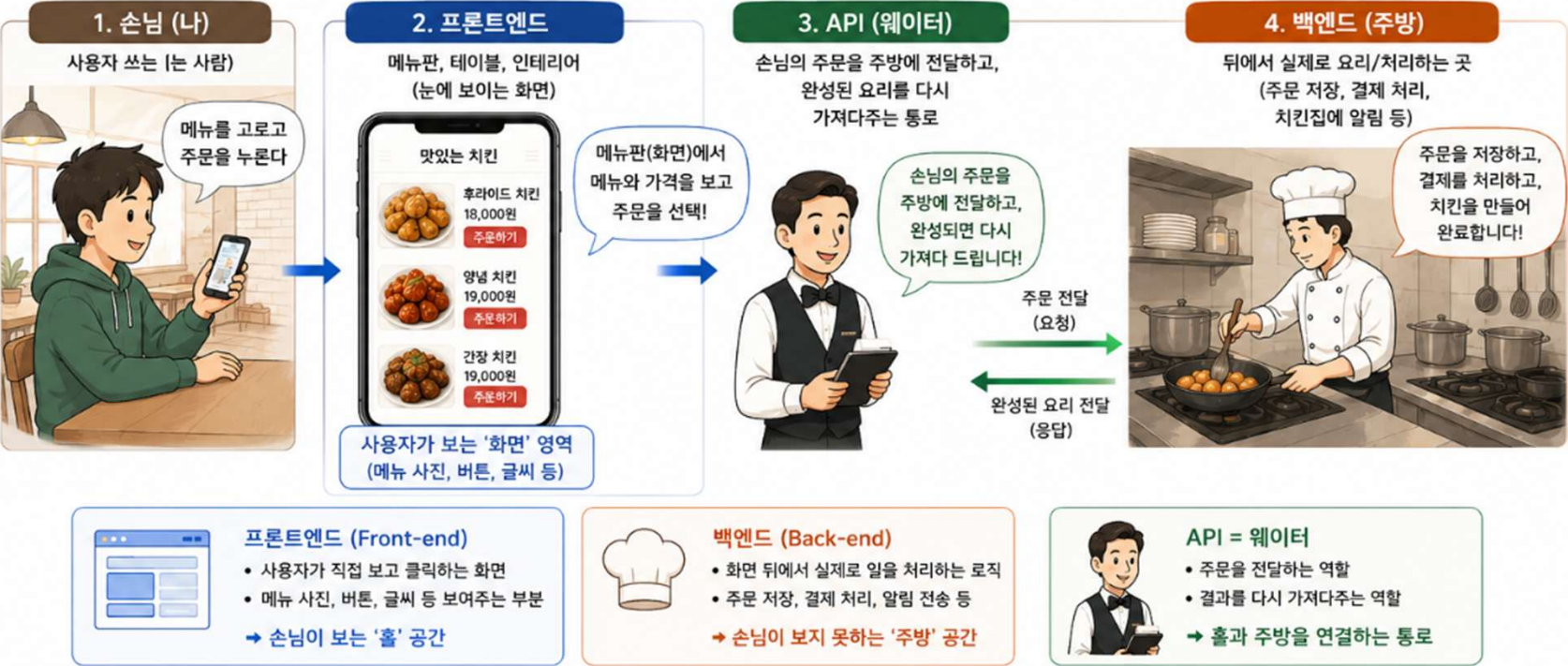


2. 프론트엔드와 백엔드의 처리 과정

레스토랑의 업무 처리 과정으로 비유

레스토랑	 손님 (나)	 메뉴판, 테이블, 인테리어	 웨이터	 주방
IT 서비스	사용자 (앱을 쓰는 사람)	프론트엔드 (눈에 보이는 화면)	API (손님의 주문을 주방에 전달하고, 완성된 요리를 다시 가져다주는 통로)	백엔드 (뒤에서 실제로 요리/처리하는 곳)



3. 백엔드 개발 언어와 프레임워크

대표 백엔드 언어 & 프레임워크

주방(백엔드)에서 요리(데이터 처리)를 하기 위한 언어와 도구들

JavaScript (Node.js)	Python	Java
 Node.js	 Python	 Java
대표 프레임워크 Express, NestJS	대표 프레임워크 Django, FastAPI	대표 프레임워크 Spring / Spring Boot
비유  프론트엔드도 만들고 백엔드도 만드는 “만능 요리사” 한 가지 언어로 화면과 서버를 모두 다룰 수 있어 입문자에게 인기!	비유  “레시피가 잘 정리된 요리책” 문법이 쉽고 읽기 편해서 초보자도 빠르게 배움	비유  “대형 프랜차이즈 주방 시스템” 크고 안정적인 서비스 (은행, 대기업)에서 많이 사용
특징 • 웹 전체를 JavaScript 하나로 개발 가능 • 학습 진입 장벽이 낮고 커뮤니티가 큼 • 빠르게 개발하고 배포하기 쉬움	특징 • 문법이 간결하고 이해하기 쉬움 • 다양한 기능이 프레임워크에 잘 정리됨 • 데이터 분석, AI 등 다양한 분야에도 활용	특징 • 성능과 안정성이 뛰어남 • 대규모 시스템 구축에 적합 • 기업 환경에서 표준으로 널리 사용

★ 핵심은 언어 자체가 아니라 개념!

같은 김치찌개도 어떤 주방장은 가스레인지로, 어떤 주방장은 인덕션으로 끓이지만 결과물(요리)은 같은 것처럼, 언어가 달라도 결국 하는 일(데이터 처리)은 비슷합니다.



4. API

API (Application Programming Interface) = “주방과 손님 사이의 주문 규칙”



대표적인 API 방식 비교

1. RESTful API

비유: 정해진 메뉴판에서 번호로 주문
("1번 메뉴 주세요")



특징

- 가장 널리 쓰이는 방식
- URL(주소)과 동적(GET/POST 등)으로 요청을 구분
- 간단하고 이해하기 쉬움

예시

GET /api/menu/1
→ 김치찌개 정보 출력
POST /api/order
→ 주문 생성

2. GraphQL

비유: 필요한 재료만 골라서
커스텀 주문
("이 요리에서 이 재료만 빼고,
이 재료는 추가해서 딱 필요한 만큼만 주세요")



특징

- 필요한 데이터만 정확히 받을 수 있음
- 네트워크 효율이 좋고 유연함
- 초기 학습이 REST보다 조금 더 필요

예시 (요청 예시)

```
query {
  menu(id: 3) {
    name
    ingredients(exclude: ["양파"])
    addOns: ["치즈"]
    sauce(level: "half")
    rice(amount: "small")
  }
}
```

3. SOAP (참고)

비유: 아주 격식 있는 정찬 코스 주문서
(오래되고 무거운 방식)



특징

- XML 기반의 메시지 형식
- 보안 및 표준화가 강점
- 무겁고 복잡하여 사용이 감소 중
- 은행, 공공기관 등 레거시 시스템에서 아직도 사용

예시 (형식 예시)

```
<soap:Envelope>
  <soap:Header>...</soap:Header>
  <soap:Body>
    <GetMenuRequest>
      <MenuId>3</MenuId>
    </GetMenuRequest>
  </soap:Body>
</soap:Envelope>
```